АРХИТЕКТУРНЫЙ ОТЧЁТ · ЗАДАЧА SGR-49

# Аудит веток Git и интеграция глобальной архитектуры обработки ошибок

## Сравнительный анализ архитектуры: текущая ветка origin/dev и интегрированное решение feat/errors-handler

> **Цель документа:** Подробно, прозрачно и технически точно объяснить всей команде разработчиков, почему глобальная обработка исключений ( `SGR-49` ) интегрируется в актуальную архитектуру `origin/dev` , как сохраняется жизненный цикл `lifespan` и почему изменилась структура `app/main.py` .
>
> **Статус валидации:** 15 из 15 модульных и интеграционных тестов с PostgreSQL успешно пройдены (100% PASSED).

| | |
|---|---|
| **Проект** | `sistema-gestion-restaurante` (Ratatouille ERP) |
| **Функциональная ветка** | `feat/errors-handler` (Задача Jira `SGR-49` ) |
| **Целевая ветка интеграции** | `origin/dev` (Базовый коммит `daac6dd` - PR #9) |
| **Технологический стек** | Python 3.12 · FastAPI 0.162 · SQLAlchemy 2.1 · PostgreSQL 15 · Pydantic v2 |
| **Дата выпуска** | 10 ноября 2026 г. |
| **Целевая аудитория** | Backend-разработчики, участники команды проекта и технические менторы |

## Содержание


1. Резюме для руководства и команды (Executive Summary)
2. Текущая архитектура в ветке `dev`
3. Предлагаемая архитектура ( `feat/errors-handler` , интегрированная с `dev` )
4. Архитектурное сравнение: До и После (Before vs. After)
5. Детальный разбор изменений в `app/main.py`
6. Подробный реестр изменений в остальных файлах
7. Анализ и аудит веток Git
8. Поток обработки ошибок и жизненный цикл HTTP-запроса
9. Тестирование, валидация и фактические результаты
10. Выводы и рекомендации для команды


---

# 1. Резюме для руководства и команды (Executive Summary)

### 1.1. Какую техническую проблему мы решаем?

В корпоративном backend-приложении для операционного управления рестораном (**Ratatouille ERP**) предсказуемое и безопасное поведение при возникновении ошибок так же критично, как и успешная обработка запросов. До настоящего момента в ветке `dev` существовали четыре фундаментальных архитектурных ограничения в части обработки исключений:

1. **Несогласованность формата ответов об ошибках (фрагментированный API-контракт):**

- Когда роут явно выбрасывал `HTTPException(status_code=404, detail="Plato no encontrado")` , клиент получал компактный JSON: `{"detail": "Plato no encontrado"}` .
- Однако при ошибке валидации входных данных в Pydantic v2 ( `RequestValidationError` , HTTP `422` ) FastAPI по умолчанию возвращал сложный массив технических объектов на английском языке ( `[{"type": "...", "loc": ["body", "..."], "msg": "Field required", "input": ...}]` ), раскрывая внутренние имена схем и само поле `input` с пользовательскими данными.
- При нарушении ограничений целостности PostgreSQL ( `IntegrityError` из-за дубликата уникального ключа или несуществующего внешнего ключа), если оно не было перехвачено предварительным `SELECT` , либо при непредвиденном сбое во время выполнения приложение могло вернуть нестандартизированный ответ `500` или оставить сессию SQLAlchemy в невалидном состоянии ( `PendingRollbackError` ).

1. **Сильная связность бизнес-логики с транспортным уровнем HTTP:**

- Проверки бизнес-правил выполнялись непосредственно в файлах роутов ( `app/api/dish_routes.py` , `app/api/category_routes.py` ) с выбросом `fastapi.HTTPException` . Это затрудняет повторное использование сервисного слоя ( `app/services/` ) из CLI-скриптов, фоновых задач или модульных тестов без прямой зависимости от FastAPI/Starlette.

1. **Отсутствие сквозной трассировки ( `X-Request-ID` ) и маскирования секретов в логах:**

- Запросы не имели уникального идентификатора корреляции ( `correlation_id` ), а логи не форматировались в структурированный JSON и не маскировали пароли, JWT-токены или строки подключения к БД ( `DATABASE_URL` / DSN).

1. **Расхождение схемы OpenAPI 3.1 с фактическими ответами об ошибках:**

- Интерактивная документация Swagger UI ( `/docs` ) по умолчанию отображала стандартную схему `HTTPValidationError` для кода `422` и не содержала единого каталога ответов для кодов `400` , `401` , `403` , `404` , `409` , `422` и `500` .

### 1.2. Почему исходный Pull Request вызвал конфликты слияния?

Исходный Pull Request (**PR #6**) был по ошибке открыт с целевой веткой `main` и базировался на коммите `abe6b6f` (*Merge pull request #7 from mxu-init/feat/SGR-70-allergen-model*). Параллельно с этим:

- В ветку `origin/dev` был влит **PR #9** ( `refactor/code-refinement` , коммит `daac6dd` ), затронувший 17 файлов: в нём был добавлен контекстный менеджер жизненного цикла `lifespan` с автоматической инициализацией справочных данных при старте ( `reference_service.ensure_reference_data_on_startup()` ), настроены абсолютные пути к статическим файлам ( `STATIC_DIR` ), добавлен эндпоинт `/favicon.ico` , модели SQLAlchemy разделены на отдельные модули с единым пакетом `app/models/__init__.py` , а текстовая колонка `dishes.allergens` заменена на связь «многие-ко-многим» через таблицу `dish_allergens` .
- Поскольку и PR #9 в `dev` , и ветка `feat/errors-handler` существенно изменили `app/main.py` (первый добавил `lifespan` и `STATIC_DIR` , а вторая — фабрику `create_app()` , middleware и глобальные обработчики исключений), возник прямой конфликт слияния в файле `app/main.py` .

### 1.3. Почему целевой веткой интеграции является `dev` , а НЕ `main` ?

В соответствии с принятым в команде процессом разработки (Gitflow):

- **`origin/dev`** — основная интеграционная ветка, в которой объединяются и тестируются все завершённые задачи текущего спринта (включая справочник аллергенов SGR-70 и рефакторинг архитектуры из PR #9).
- **`origin/main`** предназначена исключительно для стабильных релизов. Более того, проведённый аудит показал, что в `origin/main` напрямую попали два коммита ( `266af6a` и `daeb2d2` ), относящиеся к задаче CRUD столов, для которой уже открыт отдельный Pull Request в `dev` (**PR #10**, `feature/sgr-23-tables-crud` → `dev` ).
- Следовательно, слияние в `main` нарушило бы ветвление проекта и привело бы к обходу интеграционного тестирования в `dev` .

### 1.4. Предлагаемые архитектурные улучшения и итоговый результат

Интеграция ветки `feat/errors-handler` поверх актуального состояния `origin/dev` обеспечивает:

- **Единый контракт ошибок на испанском языке ( `ErrorResponse` ):** Любая ошибка `4xx` или `5xx` (включая ошибки валидации Pydantic `422` и ошибки PostgreSQL) возвращает строго `{"detail": "Понятное сообщение на испанском языке"}` .
- **Полное сохранение функциональности `dev` :** Без малейших потерь сохранены `lifespan` ( `ensure_reference_data_on_startup` ), `STATIC_DIR` , `/favicon.ico` , `/docs/favicon.ico` , `/health` , `/health/db` и все существующие роутеры ( `dishes` , `categories` , `allergens` , `test` ).
- **Транспортно-независимая иерархия доменных исключений ( `AppError` ):** Позволяет сервисам выбрасывать семантические исключения ( `NotFoundError` , `ConflictError` , `DomainValidationError` и др.) без импорта FastAPI при сохранении 100% обратной совместимости с существующими роутами, использующими `HTTPException` .
- **Автоматическая трансляция ошибок PostgreSQL ( `SQLSTATE` ) и гарантированный `session.rollback()` :** Предотвращает утечку деталей SQL-запросов и защищает сессии SQLAlchemy от блокировки в состоянии `PendingRollbackError` .
- **Чистый ASGI Middleware ( `CorrelationIdMiddleware` ) и структурированное JSON-логирование с маскированием секретов ( `[REDACTED]` ).**

---

## 2. Текущая архитектура в ветке `dev`

Перед внесением изменений был детально проанализирован исходный код ветки `origin/dev` (коммит `daac6dd`). Ниже описано фактическое устройство приложения в `dev`.

### 2.1. Инициализация приложения и роль `app/main.py`

В `origin/dev` файл `app/main.py` состоит из 50 строк и выполняет всю настройку приложения непосредственно в глобальной области видимости модуля (во время импорта файла):

```
BASE_DIR = Path(__file__).resolve().parent.parent
STATIC_DIR = BASE_DIR / "static"

@asynccontextmanager
async def lifespan(app: FastAPI):
    reference_service.ensure_reference_data_on_startup()
    yield

app = FastAPI(
    title=settings.APP_NAME,
    version=settings.APP_VERSION,
    description=settings.APP_DESCRIPTION,
    lifespan=lifespan,
)

os.makedirs(STATIC_DIR / "uploads", exist_ok=True)
app.mount("/static", StaticFiles(directory=str(STATIC_DIR)), name="static")
```

### 2.2. Регистрация роутеров

В `origin/dev` файл `app/main.py` напрямую подключает четыре модуля маршрутизации к глобальному объекту `app`:

1. `test_routes.router` (`prefix="/test"`, `tags=["Test"]`)
2. `dish_routes.router` (`prefix="/dishes"`, `tags=["Platos"]`)
3. `category_routes.router` (`prefix="/categories"`, `tags=["Categorías"]`)
4. `allergen_routes.router` (`prefix="/allergens"`, `tags=["allergens"]`)

Также непосредственно в `app/main.py` объявлены четыре служебных эндпоинта:

- `GET /` → возвращает `"API Template"`.
- `GET /favicon.ico` и `GET /docs/favicon.ico` → возвращают `FileResponse(STATIC_DIR / "favicon.ico")`.
- `GET /health` → возвращает `{"status": "ok"}`.
- `GET /health/db` → выполняет `SELECT 1` через `engine` SQLAlchemy для проверки соединения с PostgreSQL.

### 2.3. Управление запуском (`lifespan`) и справочными данными

Одним из ключевых улучшений в `origin/dev` (внесённым в PR #9) стала система управления базовыми справочниками:

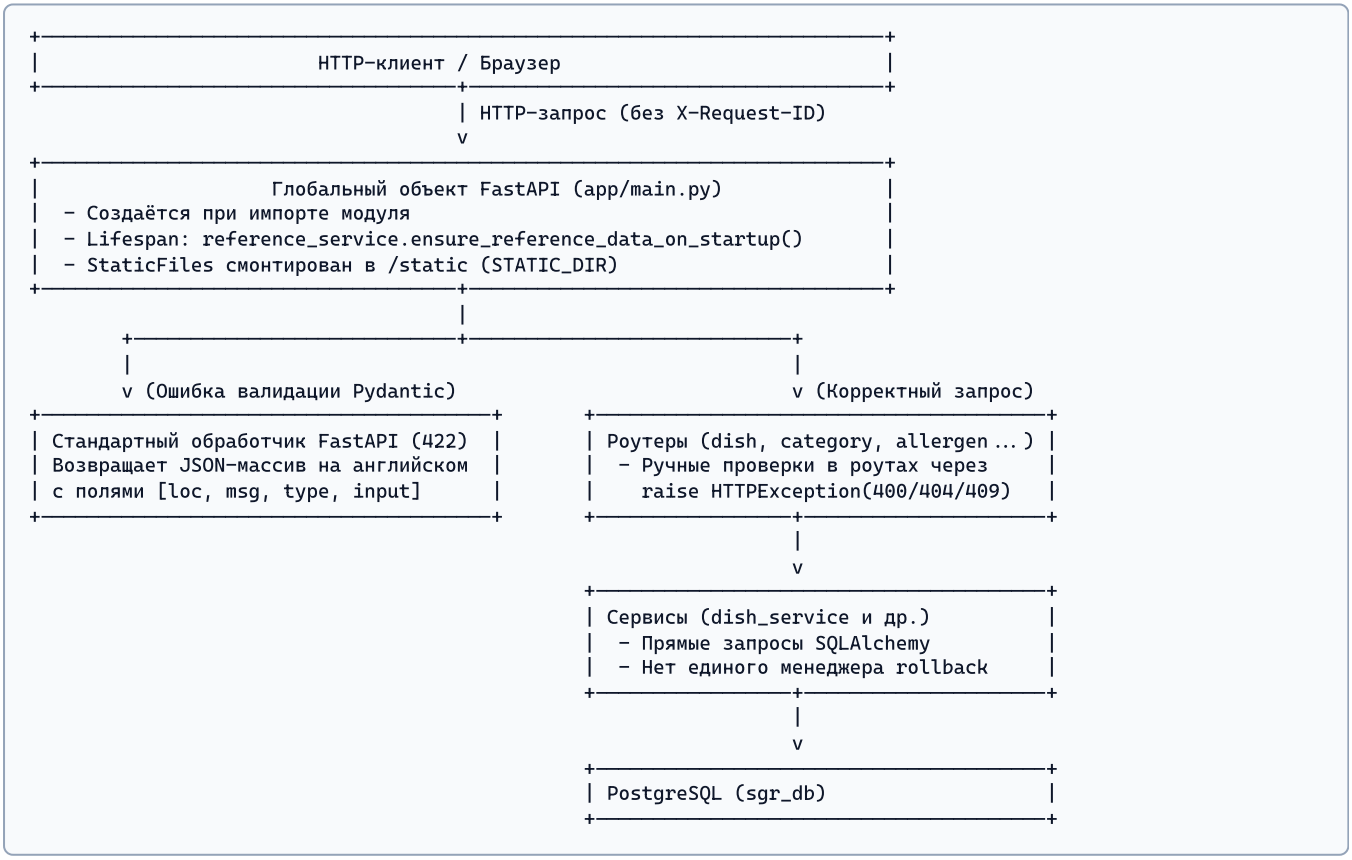
- **`app/core/reference_data.py`**: Определяет единый источник истины для 5 базовых категорий меню (`BASE_CATEGORIES`: *Entrantes, Primeros, Segundos, Postres, Bebidas*) и 14 обязательных аллергенов ЕС (`EU_ALLERGENS`, Регламент ЕС 1169/2011).
- **`app/services/reference_service.py`**: Реализует функцию `ensure_reference_data_on_startup()`. При запуске FastAPI внутри `lifespan` она проверяет наличие таблиц `categories` и `allergens` через

`sqlalchemy.inspect(engine)`. Если миграции Alembic уже применены, функция идемпотентно дозаполняет недостающие базовые категории и аллергены в рамках одной транзакции; если таблицы ещё не созданы, выводит `logger.warning` и позволяет приложению запуститься без сбоя.

### 2.4. Middleware, обработка исключений, логирование и OpenAPI в `origin/dev`

- **Middleware:** В `origin/dev` нет пользовательских middleware. Используется только стандартный внутренний стек Starlette/FastAPI (`ServerErrorMiddleware`, `ExceptionMiddleware`, `AsyncExitStackMiddleware`).
- **Обработка HTTP-исключений и ошибок валидации:**
- В `dish_routes.py` и `category_routes.py` бизнес-проверки (длина названия, положительная цена, существование категории или аллергенов) выполняются вручную в коде роутов с вызовом `raise HTTPException( ... )` и сообщениями на испанском языке.
- Однако если Pydantic отклоняет входной JSON или параметры запроса (например, в `POST /categories/` или `PUT /dishes/{id}`), срабатывает стандартный обработчик FastAPI, возвращающий англоязычный массив ошибок Pydantic (`loc`, `msg`, `type`, `input`).
- Глобальные обработчики для `SQLAlchemyError` и непредвиденных исключений `Exception` отсутствуют.
- **Логирование и трассировка:**
- Централизованная конфигурация логирования отсутствует. В `database_connection.py` включён `echo=True` на уровне `engine`, а в `reference_service.py` используется стандартный `logging.getLogger(__name__)`.
- У HTTP-запросов нет заголовка `X-Request-ID` или идентификатора корреляции.
- **OpenAPI:**
- Используется стандартная генерация схемы FastAPI без переопределения ответов `422` и без единой модели ошибок.

### 2.5. Диаграмма текущей архитектуры в `origin/dev`



## 3. Предлагаемая архитектура ( `feat/errors-handler` , интегрированная с `dev` )

Интегрированная архитектура объединяет все наработки `origin/dev` ( `lifespan` , справочные данные, модульные модели, абсолютные пути к статике) с централизованной подсистемой обработки ошибок, трассировки и безопасности из `feat/errors-handler` ( `SGR-49` ).

### 3.1. Фабрика приложения `create_app()` и сохранение `lifespan`

Вместо разрозненной настройки глобального объекта `app` при импорте модуля, `app/main.py` инкапсулирует сборку приложения в функции `create_app() → FastAPI` , а в последней строке файла создаёт экземпляр `app = create_app()` , благодаря чему команда запуска `uvicorn app.main:app --reload` работает без каких-либо изменений.

Внутри `create_app()` выполняются следующие шаги:

1. Инициализируется структурированный JSON-логгер ( `configure_logging()` ).
2. Создаётся экземпляр `FastAPI` , в который передаются `lifespan=lifespan` (вызывающий `reference_service.ensure_reference_data_on_startup()` ) и каталог ответов `responses=COMMON_ERROR_RESPONSES` .
3. Регистрируются `CorrelationIdMiddleware` и глобальные обработчики исключений через `register_exception_handlers(application)` .
4. Создаётся директория `STATIC_DIR / "uploads"` и монтируется `/static` с использованием абсолютного пути `STATIC_DIR` .
5. Подключаются все роутеры из `dev` ( `test_routes` , `dish_routes` , `category_routes` , `allergen_routes` ) и демонстрационный роутер контрактов ошибок ( `routes.router` ).
6. Регистрируются служебные маршруты `/` , `/favicon.ico` , `/docs/favicon.ico` , `/health` и `/health/db` .
7. Вызывается `configure_openapi(application)` для приведения схемы OpenAPI 3.1 в полное соответствие с моделью `ErrorResponse` .

### 3.2. Критическая архитектурная оценка: Преимущества, компромиссы и обоснование

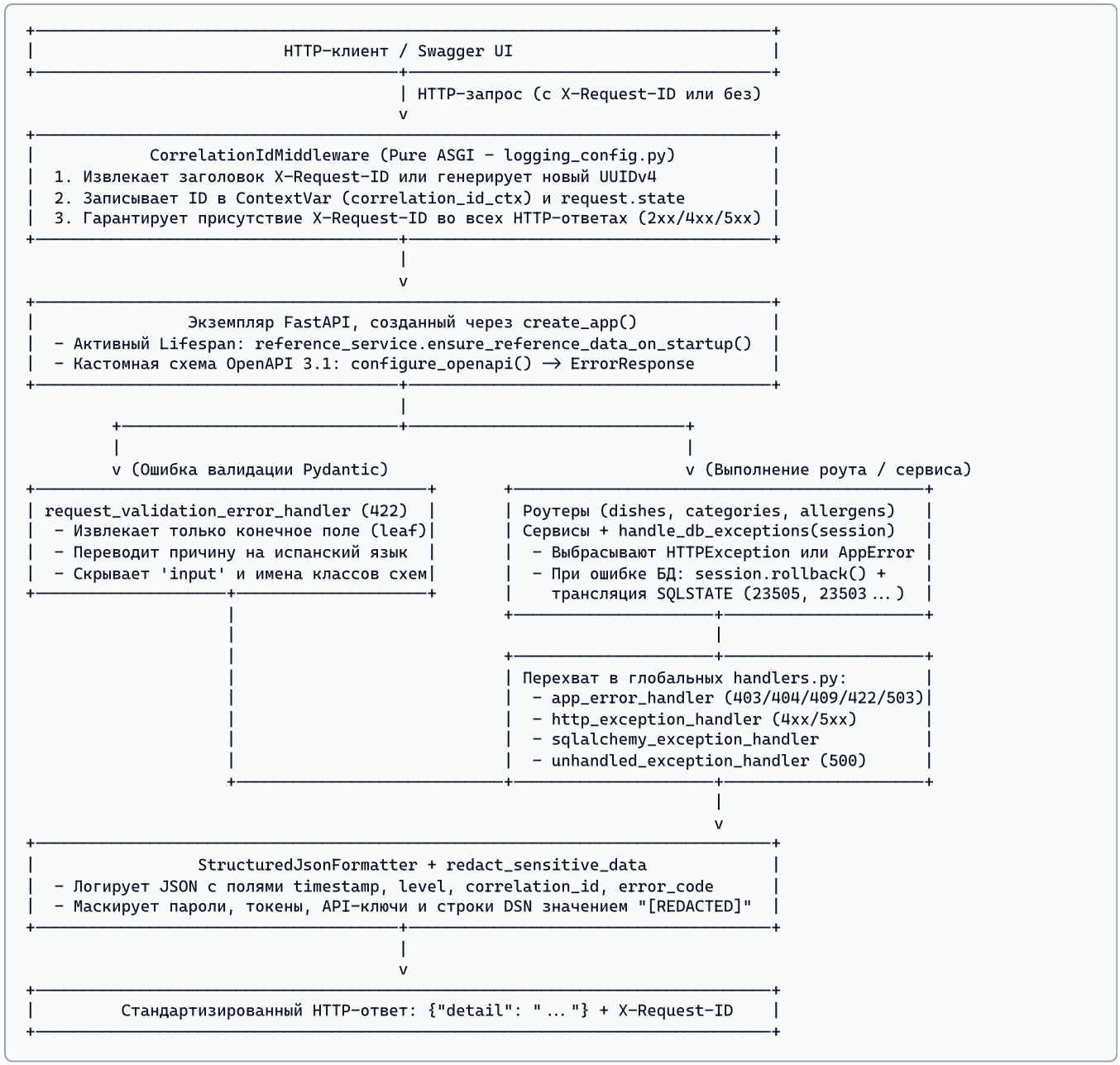
Как инженеры-архитекторы, мы не должны считать любое усложнение кода автоматическим улучшением. Оценим объективно каждое архитектурное решение, привносимое веткой `feat/errors-handler` :

- **Оправдано ли использование фабрики `create_app()` для проекта текущего масштаба?**
- *Преимущества:* Позволяет создавать изолированные экземпляры приложения в автотестах без разделяемого глобального состояния и побочных эффектов при импорте; делает порядок инициализации явным и детерминированным (логирование → FastAPI с lifespan → middleware → обработчики исключений → статика → роутеры → OpenAPI).
- *Недостатки / когнитивная нагрузка:* Добавляет один уровень абстракции по сравнению с плоским скриптом из базовых примеров FastAPI.
- *Вердикт:* **Полностью оправдано**, так как наличие строки `app = create_app()` в конце модуля сохраняет 100% совместимость с привычным запуском через Uvicorn, одновременно повышая тестируемость.
- **Как `create_app()` взаимодействует с `lifespan` ?**
- Нативно и без конфликтов: асинхронный контекстный менеджер `lifespan` передаётся в конструктор `FastAPI( ... , lifespan=lifespan)` внутри `create_app()` . При запуске сервера Uvicorn или при использовании `with TestClient(app) as client:` в тестах Starlette автоматически вызывает `ensure_reference_data_on_startup()` .
- **Насколько корректна реализация `CorrelationIdMiddleware` как Pure ASGI Middleware?**

- *Ключевое преимущество:* Стандартный класс `BaseHTTPMiddleware` в Starlette имеет известные накладные расходы и особенности работы с `ContextVar` при пробросе исключений. Реализация `CorrelationIdMiddleware` напрямую через интерфейс ASGI ( `__call__(self, scope, receive, send)` ) — это образцовое техническое решение: оно считывает или генерирует `X-Request-ID` , сохраняет его в `correlation_id_ctx` ( `ContextVar` ) и `scope["state"]["correlation_id"]` , гарантированно добавляет заголовок в `http.response.start` и очищает контекст в блоке `finally` .
- **Какие элементы требуют внимания или являются потенциально избыточными?**

1. **Демо-роутер `app/api/routes.py` ( `/errores-demo` ):** Содержит 5 эндпоинтов для интерактивной проверки контрактов ошибок ( `403` , `404` , `409` , `422` , `500` ) прямо из Swagger UI. Для приёмки задачи `SGR-49` это очень удобно, однако в будущем при подготовке промышленного релиза его стоит либо включать только при `settings.DEBUG == True` , либо оставить исключительно в тестовых фикстурах.
2. **Защита эндпоинта `/favicon.ico` :** В ветке `origin/dev` функция `favicon()` безусловно возвращала `FileResponse(STATIC_DIR / "favicon.ico")` . Но сам файл `static/favicon.ico` отсутствует в Git-репозитории. При включении глобального обработчика исключений отсутствие файла на диске вызывает `RuntimeError` внутри Starlette, что приводило к ответу `500` . В ходе интеграционного тестирования мы выявили эту проблему и добавили проверку `if favicon_path.is_file(): return FileResponse(favicon_path)` , возвращая корректный статус `204 No Content` , если файл иконки отсутствует.

### 3.3. Диаграмма интегрированной архитектуры



## 4. Архитектурное сравнение: До и После (Before vs. After)

В таблице ниже представлено системное сравнение текущей архитектуры в `origin/dev` и предлагаемой архитектуры после интеграции `feat/errors-handler`:

| Область архитектуры | Текущая реализация в `origin/dev` | Предлагаемая архитектура (`feat/errors-handler` + `dev`) | Техническое преимущество для команды |
|---|---|---|---|
| **Инициализация приложения (`app/main.py`)** | Прямое создание глобального объекта `app = FastAPI(...)` при импорте модуля. | Фабрика приложения `create_app() → FastAPI` с сохранением `app = create_app()` в конце файла. | Обеспечивает изоляцию тестов без побочных эффектов при импорте и структурирует сборку приложения, не ломая запуск `uvicorn app.main:app`. |
| **Жизненный цикл (`lifespan`) и справочники** | Контекстный менеджер `lifespan` вызывает `ensure_reference_data_on_startup()`. | **Полностью сохранено**: `create_app()` передаёт `lifespan=lifespan` в конструктор `FastAPI`. | Гарантирует наличие 5 базовых категорий и 14 аллергенов ЕС при каждом запуске сервера и в интеграционных тестах. |
| **Контракт ответа об ошибке** | Смешанный: `{"detail": "..."}` для `HTTPException`, но англоязычный массив `[{"loc", "msg", "type", "input"}]` для ошибок `422`. | Единый строгий контракт `ErrorResponse`: `{"detail": "Сообщение на испанском"}` для **всех** ответов `4xx` и `5xx`. | Frontend-разработчики и внешние клиенты всегда обрабатывают единую предсказуемую структуру JSON. |
| **Ошибки валидации Pydantic v2 (`422`)** | Стандартные сообщения Pydantic на английском; раскрытие пути вложенности (`body → usuario → ...`) и поля `input`. | `translate_validation_errors()` переводит более 50 типов ошибок Pydantic v2 на испанский и извлекает только имя конечного поля (`extract_field_name`). | Понятные пользователю сообщения (`"Error en el campo 'nombre': El campo es obligatorio"`) и отсутствие утечки внутренних схем. |
| **Доменные исключения vs. HTTP** | Роуты и сервисы напрямую зависят от `fastapi.HTTPException`. | Иерархия `AppError` (`NotFoundError`, `ConflictError`, `PermissionDeniedError`, `DomainValidationError`, `DatabaseRetryableError`, `InfrastructureError`) + полная поддержка `HTTPException`. | Отвязывает бизнес-логику (`services/`) от веб-фреймворка без необходимости переписывать уже существующие роуты. |

| Область архитектуры | Текущая реализация в `origin/dev` | Предлагаемая архитектура (`feat/errors-handler` + `dev`) | Техническое преимущество для команды |
|---|---|---|---|
| **Ошибки БД (PostgreSQL / SQLAlchemy)** | Нет глобального перехвата. Необработанное нарушение ограничений БД приводит к ошибке `500` или блокировке сессии. | `translate_sqlalchemy_error()` сопоставляет коды `SQLSTATE` (`23505`, `23503`, `23502`, `23514`, `40001`, `40P01`), а `handle_db_exceptions` выполняет `session.rollback()`. | Преобразует конфликты уникальности и внешних ключей в понятные ответы `409` / `422` на испанском языке и предотвращает `PendingRollbackError`. |
| **Непредвиденные исключения (`500`)** | Стандартный обработчик Starlette. | `unhandled_exception_handler` перехватывает любые `Exception`, логирует стектрейс внутри сервера и возвращает клиенту `{"detail": "Error interno del servidor"}`. | Полностью исключает утечку стектрейсов Python, SQL-запросов и паролей БД в ответах клиенту (требование OWASP). |
| **Трассировка запросов (`X-Request-ID`)** | Отсутствует. | `CorrelationIdMiddleware` (Pure ASGI) считывает или генерирует `X-Request-ID` и связывает его с `ContextVar`. | Позволяет мгновенно найти в логах сервера все записи, относящиеся к конкретному запросу клиента, по заголовку `X-Request-ID`. |
| **Логирование и защита секретов** | Обычный текстовый вывод без фильтрации секретов. | `StructuredJsonFormatter` + `redact_sensitive_data()`, заменяющая пароли, токены и строки DSN на `[REDACTED]`. | Структурированные JSON-логи защищены от случайного попадания паролей и ключей API в консоль или систему сбора логов. |
| **Документация OpenAPI 3.1 (`/docs`)** | Отображает `HTTPValidationError` для всех маршрутов с входными параметрами. | `configure_openapi()` удаляет `HTTPValidationError` и подставляет `#/components/schemas/ErrorResponse` во все ответы `422` + `COMMON_ERROR_RESPONSES`. | Документация в Swagger UI на 100% соответствует реальному поведению API. |

## 5. Детальный разбор изменений в `app/main.py`

У участников команды возник закономерный вопрос: почему файл `app/main.py` увеличился с ~50 до ~145 строк и зачем в нём появились функции `create_app()` и `configure_openapi()`? В этом разделе мы подробно и доступно объясним назначение каждого блока, чтобы у команды было полное понимание архитектурной логики.

### 5.1. За что отвечал `app/main.py` раньше в ветке `dev`?

В `origin/dev` файл `app/main.py` выполнял 5 базовых задач в виде линейного скрипта:

1. Вычислял пути к директориям (`BASE_DIR` и `STATIC_DIR`).
2. Объявлял контекстный менеджер `lifespan` для проверки базовых категорий и аллергенов при старте.
3. Создавал экземпляр `app = FastAPI(...)` и монтировал папку `/static`.
4. Подключал 4 роутера (`test`, `dish`, `category`, `allergen`).
5. Объявлял вспомогательные маршруты `/`, `/favicon.ico`, `/docs/favicon.ico`, `/health` и `/health/db`.

### 5.2. За что отвечает `app/main.py` после интеграции?

После интеграции `feat/errors-handler` модуль `app/main.py` выступает в роли **Корня композиции приложения (*Composition Root*)**. В нём нет бизнес-логики или словарей перевода ошибок (они вынесены в модули `app/core/`), но именно здесь все инфраструктурные слои собираются в единое целое в трёх чётко разделённых блоках:

1. **Управление жизненным циклом (`lifespan`, строки 31–34):**

Полностью сохранён код из `dev`, вызывающий `reference_service.ensure_reference_data_on_startup()`.

1. **Кастомизация схемы OpenAPI 3.1 (`configure_openapi`, строки 37–84):**

По умолчанию FastAPI автоматически добавляет к каждому маршруту с параметрами ответ `422 Unprocessable Entity`, ссылающийся на стандартную схему `HTTPValidationError`. Поскольку наш глобальный обработчик ошибок заменяет этот формат на единую схему `ErrorResponse` (`{"detail": "..."}`), без обновления OpenAPI-схемы документация в `/docs` вводила бы разработчиков в заблуждение. Функция `configure_openapi()` удаляет устаревшие схемы `HTTPValidationError` / `ValidationError` и перенаправляет все ответы `422` на `#/components/schemas/ErrorResponse`.

1. **Фабрика приложения (`create_app`, строки 87–142) и глобальный экземпляр (`app = create_app()`, строка 145):**

Конфигурирует логирование, создаёт экземпляр `FastAPI` (с `lifespan` и `COMMON_ERROR_RESPONSES`), подключает `CorrelationIdMiddleware`, регистрирует 5 глобальных обработчиков исключений (`register_exception_handlers`), монтирует `STATIC_DIR` и регистрирует все роутеры и служебные эндпоинты.

### 5.3. Зачем нужна фабрика приложения `create_app()`?

Когда объект `app = FastAPI()` создаётся и настраивается прямо в глобальной области видимости модуля, любой тестовый файл при импорте `from app.main import ...` немедленно выполняет весь глобальный код. Использование функции-фабрики `create_app()` даёт два важных преимущества:

- В автотестах можно вызывать `create_app()`, получая чистый, изолированный экземпляр приложения для каждого теста без взаимного влияния тестов друг на друга.
- Поскольку в последней строке файла остаётся `app = create_app()`, **способ запуска приложения для команды вообще не меняется**: команда `uvicorn app.main:app --reload` работает точно так же, как и раньше.

### 5.4. Почему middleware и обработчики исключений регистрируются централизованно?

Если бы перехват `SQLAlchemyError`, перевод ошибок Pydantic или генерация заголовка `X-Request-ID` выполнялись вручную внутри каждого файла маршрутов (`dish_routes.py`, `category_routes.py`, `table_routes.py` и т.д.), это привело бы к массовому дублированию кода, а любая забытая проверка оборачивалась бы необработанной ошибкой `500`. Централизованная регистрация в `create_app()` через `application.add_middleware(CorrelationIdMiddleware)` и `register_exception_handlers(application)` **автоматически защищает все существующие и будущие роутеры проекта**.

### 5.5. Разделение ответственности против неоправданного усложнения

Важно чётко разграничивать **случайную сложность** (переусложнение кода без практической пользы) и **архитектурное разделение ответственности**:

- В `app/main.py` **нет** регулярных выражений, таблиц перевода SQLSTATE или логики отката транзакций — вся эта функциональность аккуратно изолирована в профильных модулях (`app/core/exceptions.py`, `app/core/error_translations.py`, `app/core/handlers.py`, `app/core/logging_config.py`, `app/schemas/errors.py`).
- Увеличение объёма `app/main.py` обусловлено исключительно функцией `configure_openapi()` (~45 строк), необходимой для корректного отображения `ErrorResponse` в Swagger UI, и обёрткой `create_app()`.

---

## 6. Подробный реестр изменений в остальных файлах

Ниже приведено детальное описание всех созданных и изменённых файлов в рамках интеграции `feat/errors-handler` с веткой `origin/dev`, включая их прежнее состояние, суть изменений, обоснование и влияние на другие модули.

### 6.1. Сводная таблица затронутых файлов

| Файл | Статус относительно `origin/dev` | Основное назначение | Влияние на совместимость |
|---|---|---|---|
| `app/main.py` | **Изменён (Интегрирован)** | Объединяет `create_app()`, `configure_openapi()`, `CorrelationIdMiddleware` и `register_exception_handlers()` с `lifespan`, `STATIC_DIR` и `/favicon.ico` из `dev`. | 100% совместимость с `uvicorn app.main:app --reload` и всеми маршрутами `dev`. |
| `app/core/exceptions.py` | **Новый** | Определяет базовый класс `AppError` и 6 доменных подклассов, не зависящих от HTTP-транспорта (`403`, `404`, `409`, `422`, `500`, `503`). | Не конфликтует с существующим кодом; доступен для импорта в любых сервисах. |
| `app/core/error_translations.py` | **Новый** | Переводит ошибки Pydantic v2 и коды `SQLSTATE` PostgreSQL на испанский язык; предоставляет контекстные менеджеры `handle_db_exceptions` и `handle_async_db_exceptions` с гарантированным `rollback()`. | Полностью совместим с синхронными сессиями (`Session`) в `dev` и будущими асинхронными сессиями. |
| `app/core/handlers.py` | **Новый** | Реализует 5 глобальных обработчиков исключений (`AppError`, `RequestValidationError`, `StarletteHTTPException`, `SQLAlchemyError`, `Exception`). | Сохраняет поведение существующих `HTTPException` на испанском языке и переводит стандартные английские статусы (`Not Found`, `Unauthorized`). |
| `app/core/logging_config.py` | **Новый** | Настраивает JSON-логирование (`StructuredJsonFormatter`), маскирование секретов (`redact_sensitive_data`) и `CorrelationIdMiddleware`. | Прозрачно для всех роутов; автоматически добавляет заголовок `X-Request-ID`. |

| Файл | Статус относительно `origin/dev` | Основное назначение | Влияние на совместимость |
|---|---|---|---|
| `app/schemas/errors.py` | **Новый** | Определяет строгую Pydantic-модель `ErrorResponse(detail: str)` с `extra="forbid"` и словарь `COMMON_ERROR_RESPONSES`. | Унифицирует схему ошибок в OpenAPI 3.1. |
| `app/core/config.py` | **Изменён** | Добавляет `extra="ignore"` в `SettingsConfigDict(env_file=".env", extra="ignore")` и аннотацию типа для `database_url`. | Предотвращает падение приложения при старте, если в локальном `.env` присутствуют дополнительные переменные ( `DEBUG`, `SECRET_KEY`, `JIRA_*` ). |
| `app/api/routes.py` | **Новый** | Демонстрационный роутер `/errores-demo` для интерактивной проверки всех типов доменных ошибок и вложенной валидации в Swagger UI. | Изолирован под префиксом `/errores-demo`; не пересекается с `/dishes`, `/categories`, `/allergens` и `/test`. |
| `scripts/init_db.py` | **Новый** | Безопасный скрипт инициализации БД, подключающийся к системной базе `postgres` в режиме `AUTOCOMMIT` и создающий `sgr_db`, если она ещё не существует. | Упрощает первичное развёртывание проекта перед запуском `alembic upgrade head`. |
| `tests/test_exception_handlers.py` | **Новый** | 11 модульных и контрактных тестов для доменных исключений, валидации Pydantic 422, схемы OpenAPI, отката сессии SQLite в памяти и маскирования секретов. | Не требует внешних сервисов; выполняется менее чем за 0.5 с. |
| `tests/test_db_exceptions.py` | **Новый (Адаптирован под `dev` )** | 4 интеграционных теста с реальной БД PostgreSQL ( `23505`, `23503`, восстановление сессии после rollback и комплексная проверка `create_app()` + `lifespan` + роутов `dev` ). | Обновлён для импорта `CategoryModel` и `DishModel` из `app.models` с учётом удаления колонки `dishes.allergens`. |
| `pytest.ini` | **Новый** | Задаёт `testpaths = tests` и `python_files = test_*.py`. | Исключает ошибочное сканирование файла `app/models/test_model.py` при запуске `pytest` без аргументов (устраняет `InvalidRequestError` в SQLAlchemy). |

| Файл | Статус относительно `origin/dev` | Основное назначение | Влияние на совместимость |
|---|---|---|---|
| `.env.example` / `.gitignore` / `README.md` | **Новые / Изменённые** | Шаблон `.env.example`, правила `.gitignore` и документация по архитектуре ошибок `SGR-49`, объединённая с инструкциями по Alembic и `seed_data` из `dev`. | Улучшает документацию и упрощает настройку окружения для всей команды. |

### 6.2. Детали критической адаптации в `tests/test_db_exceptions.py`

При проверке совместимости `feat/errors-handler` с последними изменениями в `origin/dev` (PR #9) мы обнаружили два важных структурных изменения в моделях `dev`, напрямую затрагивавших `tests/test_db_exceptions.py`:

1. В `origin/dev` модель `CategoryModel` была вынесена в отдельный файл `app/models/category_model.py` и экспортирована через `app/models/__init__.py`.
2. В `origin/dev` (миграция `c23d2bf3b75a`) текстовая колонка `allergens` была удалена из `DishModel` и заменена связью через промежуточную таблицу `dish_allergens`.

**Выполненное действие:** Мы обновили `tests/test_db_exceptions.py`, заменив импорт на `from app.models import CategoryModel, DishModel`, убрали устаревший параметр `allergens=payload.allergens` при создании `DishModel` и добавили новый интеграционный тест `test_create_app_lifespan_reference_data_and_dev_endpoints`, проверяющий полный цикл запуска `create_app()` вместе с `lifespan` и маршрутами `dev`.

---

## 7. Анализ и аудит веток Git

Был проведён полный технический аудит локального и удалённого Git-репозитория ( `origin/dev` , `origin/main` , `feat/errors-handler` , а также Pull Requests с #1 по #10).

### 7.1. Топология веток и общая база слияния ( `merge-base` )

Общим предком ( `git merge-base` ) для всех трёх веток до начала нашей интеграции являлся коммит `abe6b6f2a3d7db66803c6b9b2b07b35fe9360c95` (*Merge pull request #7 from mxu-init/feat/SGR-70-allergen-model*):

```
                     +-- 266af6a (fix: CRUD table) -- daeb2d2 (first commit) [origin/main]
                    /    [Также открыт как PR #10: feature/sgr-23-tables-crud → dev]
                   /
--- abe6b6f (Base) +——— b9870ec .. f0126c5 -- daac6dd (Merge PR #9: refactor/code-refinement) [origin/dev]
                    \                                                          \
                     +--- b6643ef .. 069b0c4 (SGR-49 Error Handling) ————————————+——> [feat/errors-handler Инте
```

### 7.2. Что находится в `origin/dev` ( `abe6b6f..daac6dd` )?

Ветка `origin/dev` включает **8 коммитов из PR #9 ( `refactor/code-refinement` )**:

- `b9870ec` : Выделение `CategoryModel` в отдельный модуль `app/models/category_model.py` и создание пакета `app/models/__init__.py` .
- `dd33225` : Удаление устаревшей текстовой колонки `allergens` из `DishModel` и добавление миграции `c23d2bf3b75a` .
- `bbc68c8` : Привязка аллергенов к блюдам через промежуточную таблицу `dish_allergens` ( `allergen_ids` в `dish_routes.py` , `dish_service.py` , `allergen_service.py` ).
- `5666013` : Создание `app/core/reference_data.py` , `app/services/reference_service.py` , миграции `aacc49216385` (заполняющей 5 базовых категорий и 14 аллергенов ЕС) и добавление `lifespan` в `app/main.py` .
- `1730b1e` : Исключение шаблонной модели `TestModel` из экспортов `app/models/__init__.py` .
- `a1c57b2` : Очистка дублирующейся логики в `scripts/seed_data.py` .
- `3ce4a5b` : Обновление `README.md` (актуализация структуры репозитория и инструкции по `alembic upgrade head` ).
- `f0126c5` : Добавление абсолютного пути `STATIC_DIR` и эндпоинтов `/favicon.ico` и `/docs/favicon.ico` в `app/main.py` .

### 7.3. Какие изменения есть в `origin/main` , но отсутствуют в `origin/dev` ?

При выполнении `git log origin/dev..origin/main` обнаружено ровно **2 коммита**, присутствующих в `origin/main` поверх `abe6b6f` :

| SHA коммита | Автор и дата | Сообщение коммита | Затронутые файлы | Анализ релевантности и архитектурное решение |
|---|---|---|---|---|
| `266af6a` | `moigarcia00` (07.11.2026) | `fix: CRUD table` | `alembic/env.py`, 2 миграции ( `8c938a804c11`, `71ad42bf89a7` ), `app/api/table_routes.py`, `app/main.py`, `app/models/table_model.py`, `app/schemas/table_schema.py`, `app/services/table_service.py` | **НЕ включать в `feat/errors-handler`.** Этот коммит реализует CRUD столов ( `SGR-23` ), который был ошибочно запушен в `main`, но уже оформлен автором как отдельный открытый Pull Request в `dev` (**PR #10**: `feature/sgr-23-tables-crud` → `dev`, тот же SHA `266af6a` ). Кроме того, его миграция `8c938a804c11` базируется на ревизии `76ea12b38f94`, что привело бы к раздвоению истории миграций (*multiple heads*) в Alembic относительно ревизии `aacc49216385` в `dev`. Эта функциональность должна пройти штатное ревью и слияние в рамках PR #10. Примечание: в нашем словаре `CONSTRAINT_SPANISH_MESSAGES` ( `error_translations.py` ) уже предусмотрена поддержка ограничения уникальности номера стола ( `"uq_tables_number": "Ya existe una mesa con ese número"` ). |

| SHA коммита | Автор и дата | Сообщение коммита | Затронутые файлы | Анализ релевантности и архитектурное решение |
|---|---|---|---|---|
| `daeb2d2` | `moigarcia00` (10.11.2026) | `first commit` | `README.md` (+1 строка `# MYWARFRAME` в самом конце файла) | **Нерелевантно / тестовый мусор.** Случайный тестовый коммит в `main`, добавляющий постороннюю строку в конец `README.md`. **Включать категорически не следует.** |

**Вывод по аудиту `main`:** Ни один из двух коммитов, отличающих `origin/main` от `origin/dev`, не должен переноситься в `feat/errors-handler`. Вся актуальная кодовая база проекта находится в `origin/dev`.

---

# 8. Поток обработки ошибок и жизненный цикл HTTP-запроса

## 8.1. Пошаговое прохождение HTTP-запроса через систему

```
[1. Входящий HTTP-запрос]
       |
       v
[2. CorrelationIdMiddleware (Pure ASGI)]
       ├── Считывает заголовок 'X-Request-ID' (или генерирует новый UUIDv4)
       ├── Сохраняет ID в ContextVar 'correlation_id_ctx' и 'request.state.correlation_id'
       |
       v
[3. Маршрутизация и валидация входных данных (FastAPI / Pydantic v2)]
       ├── Ошибка валидации типов/полей тела запроса или query-параметров?
       |     +-- ДА ──> Выбрасывается RequestValidationError
       |                ──> Перехватывается в request_validation_error_handler()
       |                ──> extract_field_name() + translate_validation_errors()
       |                ──> Логируется WARNING в JSON (с correlation_id, без поля 'input')
       |                ──> HTTP-ответ 422: {"detail": "Error en el campo 'X': ..."}
       |
       ├── Входная валидация пройдена успешно?
             +-- ДА ──> Вызов контроллера / эндпоинта / сервиса
                          |
                          +-- Сценарий A: Успешное выполнение (200 / 201 / 204)
                          |     ──> Middleware добавляет заголовок 'X-Request-ID'
                          |     ──> Клиент получает штатный JSON-ответ
                          |
                          +-- Сценарий B: Бизнес-правило выбрасывает AppError (NotFoundError, ConflictError ...)
                          |     ──> Перехватывается в app_error_handler()
                          |     ──> Сопоставляет класс исключения с HTTP-кодом (404, 409, 403, 422, 503, 500)
                          |     ──> Маскирует чувствительные данные через redact_sensitive_data()
                          |     ──> HTTP-ответ с {"detail": "<сообщение на испанском>"}
                          |
                          +-- Сценарий C: Существующий роут выбрасывает HTTPException / StarletteHTTPException
                          |     ──> Перехватывается в http_exception_handler()
                          |     ──> Сохраняет испанский текст или переводит стандартный английский detail
                          |     ──> Сохраняет заголовки (например, WWW-Authenticate) + X-Request-ID
                          |     ──> HTTP-ответ с {"detail": "<сообщение на испанском>"}
                          |
                          +-- Сценарий D: Ошибка целостности PostgreSQL (через handle_db_exceptions или глобально)
                          |     ──> Немедленно выполняет session.rollback()
                          |     ──> Извлекает 5-символьный SQLSTATE (23505, 23503 ...) и constraint_name
                          |     ──> Преобразует в ConflictError (409) или DomainValidationError (422)
                          |     ──> Никогда не раскрывает клиенту текст SQL-запроса или имена таблиц
                          |
                          +-- Сценарий E: Непредвиденное исключение (RuntimeError, KeyError, ZeroDivisionError ...)
                                ──> Перехватывается в unhandled_exception_handler()
                                ──> Логирует ERROR в JSON с внутренним типом ошибки и маскированием DSN ([REDACTED
                                ──> HTTP-ответ 500: {"detail": "Error interno del servidor"}
```

## 8.2. Конкретные примеры реальных ответов об ошибках

### Пример 1: Ошибка валидации Pydantic ( `422 Unprocessable Content` ) при отсутствии обязательного поля

- **Запрос:** `POST /errores-demo/validacion` с пустым телом `{}`
- **Заголовки ответа:**

```
HTTP/1.1 422 Unprocessable Content
content-type: application/json
x-request-id: 9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d
```

- **Тело JSON:**

```
  {
    "detail": "Error en el campo 'nombre': El campo es obligatorio; Error en el campo 'comensales': El
campo es obligatorio"
  }
```

#### Пример 2: Ошибка валидации во вложенном объекте ( `422` )

- **Запрос:** `POST /errores-demo/validacion` с телом `{"nombre": "Carlos", "comensales": 4, "usuario": {"nombre": "Carlos", "direccion": {"calle": "Gran Vía 12", "codigo_postal": "12"}}}`
- **Извлечение конечного поля:** Внутренний путь Pydantic `('body', 'usuario', 'direccion', 'codigo_postal')` автоматически очищается до конечного поля `'codigo_postal'`, не раскрывая клиенту иерархию классов ( `DireccionDemoSchema` ).
- **Тело JSON:**

```
  {
    "detail": "Error en el campo 'codigo_postal': El texto debe tener al menos 4 caracteres"
  }
```

#### Пример 3: Нарушение уникальности в PostgreSQL ( `SQLSTATE 23505` → `409 Conflict` )

- **Операция:** Попытка создать категорию с уже существующим именем ( `"Postres"` ).
- **Тело JSON, возвращаемое клиенту (без утечки SQL и имён индексов):**

```
  {
    "detail": "Ya existe una categoría con ese nombre"
  }
```

#### Пример 4: Непредвиденная серверная ошибка ( `500 Internal Server Error` ) и маскирование секретов в логах

- **Искусственный сбой:** `RuntimeError("Database failure at postgresql+psycopg://admin:supersecret@db:5432/prod")`
- **Тело JSON, возвращаемое клиенту:**

```
  {
    "detail": "Error interno del servidor"
  }
```

- **JSON-запись во внутреннем логе сервера (пароль и DSN автоматически скрыты):**

```
  {
    "timestamp": "2026-11-10T09:42:00.123456+00:00",
    "level": "ERROR",
    "logger": "sgr",
    "message": "Excepción no controlada capturada por el manejador global",
    "correlation_id": "req-corr-999",
    "context": {
      "path": "/test/crash",
      "method": "GET",
      "exception_type": "RuntimeError"
    },
    "error_code": "UNHANDLED_EXCEPTION",
    "exception_type": "RuntimeError"
  }
```

---

## 9. Тестирование, валидация и фактические результаты

Все проверки и тесты были фактически выполнены в виртуальном окружении проекта ( `.venv` , Python `3.12.10` , FastAPI `0.162.0` , SQLAlchemy `2.1.6` , Pydantic `2.14.1` ) с использованием запущенного Docker-контейнера PostgreSQL 15 ( `sgr-postgres` на порту `5433` ).

### 9.1. Выполненные команды и подтверждённые результаты

1. **Применение миграций ветки `origin/dev` к базе PostgreSQL ( `sgr-postgres` ):**

```
.\.venv\Scripts\python.exe -m alembic upgrade head
```

- **Результат:** Успешно выполнено ( `exit code 0` ). Применены ревизии `76ea12b38f94` (*create allergens tables*), `c23d2bf3b75a` (*drop allergens text column from dishes*) и `aacc49216385` (*seed base categories and allergens*).

1. **Запуск полного набора автоматических тестов ( `pytest` ):**

```
.\.venv\Scripts\python.exe -m pytest -v
```

- **Результат: 15 passed in 0.86s** ( `0 failed` , `0 errors` ).

### 9.2. Детализация 15 выполненных автотестов

| Тестовый модуль | Имя теста | Что именно проверяется | Статус |
|---|---|---|---|
| `tests/test_exception_handlers.py` | `test_not_found_error_returns_404_spanish_detail` | `NotFoundError` возвращает HTTP `404` , JSON `{"detail": "..."}` и заголовок `X-Request-ID` . | **PASSED** |
| `tests/test_exception_handlers.py` | `test_conflict_error_returns_409_spanish_detail` | `ConflictError` возвращает HTTP `409` со строгим контрактом на испанском языке. | **PASSED** |
| `tests/test_exception_handlers.py` | `test_permission_denied_error_returns_403_spanish_detail` | `PermissionDeniedError` возвращает HTTP `403` со строгим контрактом на испанском языке. | **PASSED** |
| `tests/test_exception_handlers.py` | `test_validation_missing_field_exact_prompt_contract` | `RequestValidationError` при отсутствии обязательного поля возвращает `"Error en el campo 'nombre': El campo es obligatorio"` . | **PASSED** |
| `tests/test_exception_handlers.py` | `test_validation_nested_object_extracts_leaf_field_without_schema_leak` | Извлечение имени конечного поля ( `codigo_postal` ) во вложенных объектах без утечки имён схем и префикса `body` . | **PASSED** |

| Тестовый модуль | Имя теста | Что именно проверяется | Статус |
|---|---|---|---|
| `tests/test_exception_handlers.py` | `test_unexpected_exception_returns_generic_spanish_500_without_traceback` | Исключение `RuntimeError` со строкой DSN возвращает HTTP `500` `"Error interno del servidor"` без утечки стектрейса и пароля. | **PASSED** |
| `tests/test_exception_handlers.py` | `test_openapi_schema_uses_error_response_for_422_and_removes_default_validation_schema` | Схема OpenAPI 3.1 удаляет `HTTPValidationError` и использует `#/components/schemas/ErrorResponse` во всех ответах `422`. | **PASSED** |
| `tests/test_exception_handlers.py` | `test_session_rollback_prevents_pending_rollback_error_on_reuse` | `handle_db_exceptions` вызывает `session.rollback()` и позволяет продолжить работу с той же сессией без `PendingRollbackError`. | **PASSED** |
| `tests/test_exception_handlers.py` | `test_domain_exceptions_are_transport_agnostic` | Ни один класс в иерархии `AppError` не наследуется от `fastapi.HTTPException` или `starlette.exceptions.HTTPException`. | **PASSED** |
| `tests/test_exception_handlers.py` | `test_postgresql_sqlstate_and_constraint_extraction_psycopg_and_asyncpg` | Извлечение `SQLSTATE` и `constraint_name` как из объектов драйвера `psycopg` v3, так и из `asyncpg`. | **PASSED** |
| `tests/test_exception_handlers.py` | `test_async_session_rollback_and_sensitive_redaction` | Асинхронный rollback (`handle_async_db_exceptions`), сохранение заголовка `WWW-Authenticate` при `401`, передача `X-Request-ID` и маскирование `[REDACTED]`. | **PASSED** |
| `tests/test_db_exceptions.py` | `test_sqlstate_23505_unique_violation_translates_to_409_conflict` | Интеграционный тест с реальным PostgreSQL: дубликат категории вызывает `SQLSTATE 23505` → HTTP `409` без утечки SQL. | **PASSED** |

| Тестовый модуль | Имя теста | Что именно проверяется | Статус |
|---|---|---|---|
| `tests/test_db_exceptions.py` | `test_sqlstate_23503_foreign_key_violation_translates_to_409_without_sql_leak` | Интеграционный тест с реальным PostgreSQL: создание блюда с несуществующим `category_id` вызывает `SQLSTATE 23503` → HTTP `409` без утечки SQL. | **PASSED** |
| `tests/test_db_exceptions.py` | `test_session_rollback_on_sync_and_async_handlers_prevents_pending_rollback_error` | Интеграционный тест с реальным PostgreSQL: восстановление сессии после ошибок `23505` и `23503`. | **PASSED** |
| `tests/test_db_exceptions.py` | `test_create_app_lifespan_reference_data_and_dev_endpoints` | Комплексный тест `create_app()`: проверяет выполнение `lifespan`, наличие 5 базовых категорий и 14 аллергенов ЕС, `/health`, `/health/db`, `/favicon.ico` и ответ `404` на `/dishes/999999999` с заголовком `X-Request-ID`. | **PASSED** |

### 9.3. Проблемы, обнаруженные и устранённые в ходе тестирования

1. **Конфликт сбора тестов Pytest с файлом `app/models/test_model.py`:**

- *Предсуществующая проблема в `dev`:* При запуске `pytest` без указания папки Pytest пытался импортировать файл `app/models/test_model.py` как тестовый модуль из-за префикса `test_`. Поскольку этот модуль уже был импортирован через `test_controller.py`, SQLAlchemy выбрасывал ошибку `InvalidRequestError: Table 'test' is already defined for this MetaData instance`.
- *Решение:* Добавлен конфигурационный файл `pytest.ini` с параметром `testpaths = tests`, что полностью устранило ошибку без модификации `test_model.py`.

1. **Обработка `/favicon.ico` при отсутствии файла `static/favicon.ico`:**

- *Обнаруженная проблема:* В `origin/dev` эндпоинт `favicon()` возвращал `FileResponse(STATIC_DIR / "favicon.ico")` без проверки существования файла. Так как файла `static/favicon.ico` нет в репозитории, запрос к `/favicon.ico` вызывал `RuntimeError` в Starlette.
- *Решение:* В `favicon()` добавлена проверка `if favicon_path.is_file(): return FileResponse(favicon_path)`, а при отсутствии файла возвращается корректный ответ `Response(status_code=204)`.

---

## 10. Выводы и рекомендации для команды

### 10.1. Что было улучшено и что осталось неизменным

- **Сохранено без изменений (100% совместимость с `dev`):**
- Стандартная команда запуска сервера `uvicorn app.main:app --reload`.
- Контекстный менеджер `lifespan` и автоматическая проверка/загрузка 5 базовых категорий и 14 обязательных аллергенов ЕС при старте (`reference_service.ensure_reference_data_on_startup()`).
- Модульная структура моделей в `app/models/` и связь «многие-ко-многим» между блюдами и аллергенами (`dish_allergens`).
- Все существующие эндпоинты в `dish_routes.py`, `category_routes.py`, `allergen_routes.py` и `test_routes.py`.
- **Улучшено:**
- Все ответы об ошибках (`400`, `401`, `403`, `404`, `409`, `422`, `500`, `503`) приведены к единому строгому контракту на испанском языке: `{"detail": "..."}`.
- Ошибки валидации Pydantic (`422`) больше не возвращают англоязычные технические структуры и не раскрывают поле `input`.
- Нарушения ограничений целостности PostgreSQL автоматически переводятся в понятные доменные ошибки с гарантированным вызовом `session.rollback()`.
- Все HTTP-ответы содержат заголовок трассировки `X-Request-ID`, а JSON-логи автоматически маскируют пароли, токены и строки подключения DSN.

### 10.2. На что обратить внимание при ревью Pull Request (Code Review)

При проверке PR в ветку `dev` рекомендуем коллегам обратить внимание на следующие ключевые точки:

1. **`app/main.py`:** Как фабрика `create_app()` объединяет `lifespan=lifespan` и `STATIC_DIR` из ветки `dev` с `CorrelationIdMiddleware`, `register_exception_handlers()` и `configure_openapi()`.
2. **`app/core/handlers.py` и `app/core/error_translations.py`:** Как сосуществуют существующие вызовы `HTTPException` и новые доменные исключения `AppError`, а также словари перевода ошибок валидации и ограничений БД на испанский язык.
3. **`tests/test_exception_handlers.py` и `tests/test_db_exceptions.py`:** Запустить `pytest -v` локально, чтобы убедиться в прохождении всех 15 модульных и интеграционных тестов с PostgreSQL.

### 10.3. Рекомендации по дальнейшему развитию

1. **Постепенный переход на `AppError` и `handle_db_exceptions` в новых сервисах:** Нет необходимости экстренно переписывать существующий код в `dish_routes.py` или `category_routes.py` (поскольку `http_exception_handler` полностью поддерживает существующие `HTTPException`). Однако при разработке новых модулей (бронирование столов, динамические меню, JWT-аутентификация) или после влития PR #10 (`table_routes.py`) рекомендуется использовать контекстный менеджер `with handle_db_exceptions(db):` и выбрасывать `NotFoundError` / `ConflictError` из сервисного слоя.
2. **Ограничение демо-роутера `/errores-demo` режимом отладки:** После добавления настроек окружения в `Settings` демонстрационный роутер `app/api/routes.py` (`/errores-demo`) можно подключать только при `settings.DEBUG == True`.